

Sprawozdanie z projektu programistycznego

Strava Clone

Aplikacja do śledzenia aktywności fizycznych

Projektowanie Systemów Baz Danych

Technologia backendu:	Laravel 12 (PHP 8.2+)
Technologia frontendu:	Vue 3 + TypeScript
Baza danych:	PostgreSQL 18
Repozytorium:	github.com/Console-dungeon/ strava-clone

19 maja 2026

Spis treści

1	Środowisko uruchomieniowe i konfiguracja	2
1.1	Wybór systemu zarządzania bazą danych	2
1.2	Środowisko deweloperskie	2
1.2.1	Docker / Laravel Sail	2
1.2.2	Dev Container	2
1.2.3	Haki pre-commit	3
2	Opis struktury bazy danych	4
2.1	System zarządzania bazą danych	4
2.2	Tabele aplikacyjne	4
2.2.1	Tabela <code>users</code>	4
2.2.2	Tabela <code>activities</code>	5
2.2.3	Tabela <code>activity_gpx</code>	5
2.2.4	Tabela <code>activity_logs</code>	6
2.3	Tabele systemowe Laravel	6
2.3.1	Tabela <code>sessions</code>	6
2.3.2	Tabela <code>jobs</code> i <code>failed_jobs</code>	6
2.3.3	Tabela <code>cache</code>	7
2.3.4	Tabela <code>password_reset_tokens</code>	7
2.4	Diagram relacji encji	7
2.5	Więzy integralności	7
2.5.1	Klucze obce i usuwanie kaskadowe	7
2.5.2	Ograniczenia unikalności	8
2.5.3	Indeksy	8
2.6	Iteracyjny rozwój schematu bazy danych	9
2.7	Seedery i dane testowe	10
2.7.1	DatabaseSeeder	10
2.7.2	ActivityFactory	10
2.7.3	Uruchomienie seederów	10
2.8	Testowanie i optymalizacja	11

Rozdział 1

Środowisko uruchomieniowe i konfiguracja

1.1 Wybór systemu zarządzania bazą danych

Projekt wykorzystuje **PostgreSQL 18** – relacyjny system zarządzania bazą danych (RDBMS) znany z zaawansowanego wsparcia dla typów danych (JSON, UUID, tablice), pełnej zgodności ze standardem SQL oraz wysokiej niezawodności. PostgreSQL uruchamiany jest jako kontener Docker w ramach środowiska **Laravel Sail**.

1.2 Środowisko deweloperskie

1.2.1 Docker / Laravel Sail

Środowisko lokalne opiera się na **Laravel Sail** – zestawie skryptów Docker skonfigurowanych dla Laravel. Sail uruchamia kontenery:

- **PHP/Laravel** – serwer aplikacji,
- **PostgreSQL 18** – baza danych,
- **Adminer** – graficzny interfejs bazy danych dostępny pod adresem `http://localhost:8080`,
- **Node.js** – kompilacja frontendu przez Vite.

1.2.2 Dev Container

Projekt skonfigurowany jest dla **VS Code Dev Containers** – jednorazowe otwarcie projektu w kontenerze zapewnia gotowe do pracy środowisko bez ręcznej instalacji zależności.

1.2.3 Haki pre-commit

Husky uruchamia automatycznie `pnpm lint-staged` przed każdym commitem:

- ESLint z auto-naprawą dla plików `.ts` / `.vue`,
- Prettier dla formatowania kodu,
- Laravel Pint dla PHP.

Pomijanie haków przez `-no-verify` jest zakazane.

Rozdział 2

Opis struktury bazy danych

2.1 System zarządzania bazą danych

Projekt wykorzystuje **PostgreSQL 18** uruchomiony w kontenerze Docker. Dostęp skonfigurowany jest w zmiennych środowiskowych:

- host: `pgsql`,
- port: `5432`,
- baza: `strava_clone`,
- użytkownik: `sail`.

Schemat bazy zarządzany jest przez **Laravel Migrations** – każda zmiana struktury to osobny, nieodwracalnie numerowany plik migracji.

Do zarządzania bazą danych i inspekcji danych w środowisku deweloperskim służy **Adminer** – lekki graficzny interfejs webowy uruchomiony jako osobny kontener Docker. Adminer dostępny jest pod adresem `http://localhost:8080` (port konfigurowalny przez zmienną środowiskową `ADMINER_PORT`). Interfejs skonfigurowany jest z domyślnym serwerem `pgsql` oraz ciemnym motywem wizualnym (*pepa-linha-dark*).

2.2 Tabele aplikacyjne

2.2.1 Tabela `users`

Przechowuje dane zarejestrowanych użytkowników.

Kolumna	Typ	Opis
id	BIGINT PK	Klucz główny (auto-increment)
name	VARCHAR	Imię i nazwisko użytkownika
email	VARCHAR UNIQUE	Adres e-mail (unikalny)
email_verified_at	TIMESTAMP NULL	Data weryfikacji e-mail
password	VARCHAR	Hash bcrypt hasła
garmin_email	VARCHAR NULL	E-mail do konta Garmin
garmin_password	TEXT NULL	Zaszyfrowane hasło Garmin
remember_token	VARCHAR NULL	Token „zapamiętaj mnie”
created_at	TIMESTAMP	Data rejestracji
updated_at	TIMESTAMP	Data ostatniej aktualizacji

2.2.2 Tabela activities

Główna tabela aktywności fizycznych.

Kolumna	Typ	Opis
id	BIGINT PK	Klucz główny
user_id	BIGINT FK	Klucz obcy → users.id
type	VARCHAR	Typ aktywności (running/cycling/...)
distance	FLOAT	Dystans w kilometrach
duration	INTEGER	Czas trwania w sekundach
avg_speed	FLOAT	Średnia prędkość w km/h
max_speed	FLOAT NULL	Maksymalna prędkość w km/h
avg_hr	INTEGER NULL	Średnie tętno (bpm)
max_hr	INTEGER NULL	Maksymalne tętno (bpm)
calories	INTEGER NULL	Spalone kalorie (kcal)
date	DATE	Data aktywności
notes	TEXT NULL	Notatki użytkownika
garmin_activity_id	VARCHAR NULL	ID aktywności w Garmin Connect
created_at	TIMESTAMP	Data dodania rekordu
updated_at	TIMESTAMP	Data ostatniej modyfikacji

Klucz obcy **user_id** skonfigurowany z kaskadowym usuwaniem (*ON DELETE CASCADE*).

2.2.3 Tabela activity_gpx

Przechowuje dane trasy GPS powiązanej z aktywnością.

Kolumna	Typ	Opis
id	BIGINT PK	Klucz główny
activity_id	BIGINT FK UNIQUE	Klucz obcy → <code>activities.id</code>
route_points	JSON	Tablica punktów GPS [lat, lng, ele, time]
gpx_file_path	VARCHAR NULL	Ścieżka do oryginalnego pliku GPX
elevation_gain	FLOAT NULL	Przewyższenie dodatnie w metrach
created_at	TIMESTAMP	Data dodania
updated_at	TIMESTAMP	Data modyfikacji

2.2.4 Tabela `activity_logs`

Rejestr akcji wykonanych przez użytkowników.

Kolumna	Typ	Opis
id	BIGINT PK	Klucz główny
user_id	BIGINT FK	Klucz obcy → <code>users.id</code>
action	VARCHAR(100)	Typ akcji (stała tekstowa)
created_at	TIMESTAMP	Znacznik czasu akcji

Tabela nie posiada kolumny `updated_at` (immutable log). Indeksy założone na kolumnach `action` oraz (`user_id`, `created_at`).

2.3 Tabele systemowe Laravel

2.3.1 Tabela `sessions`

Przechowuje sesje użytkowników (driver: `database`).

Kolumna	Opis
id (PK)	Token sesji
user_id	FK do <code>users</code> (nullable)
ip_address	Adres IP klienta
user_agent	User-Agent przeglądarki
payload	Dane sesji (LONGTEXT)
last_activity	Timestamp ostatniej aktywności

2.3.2 Tabela `jobs` i `failed_jobs`

Kolejka zadań asynchronicznych (driver: `database`). Tabela `jobs` przechowuje oczekujące zadania, `failed_jobs` rejestruje nieudane próby.

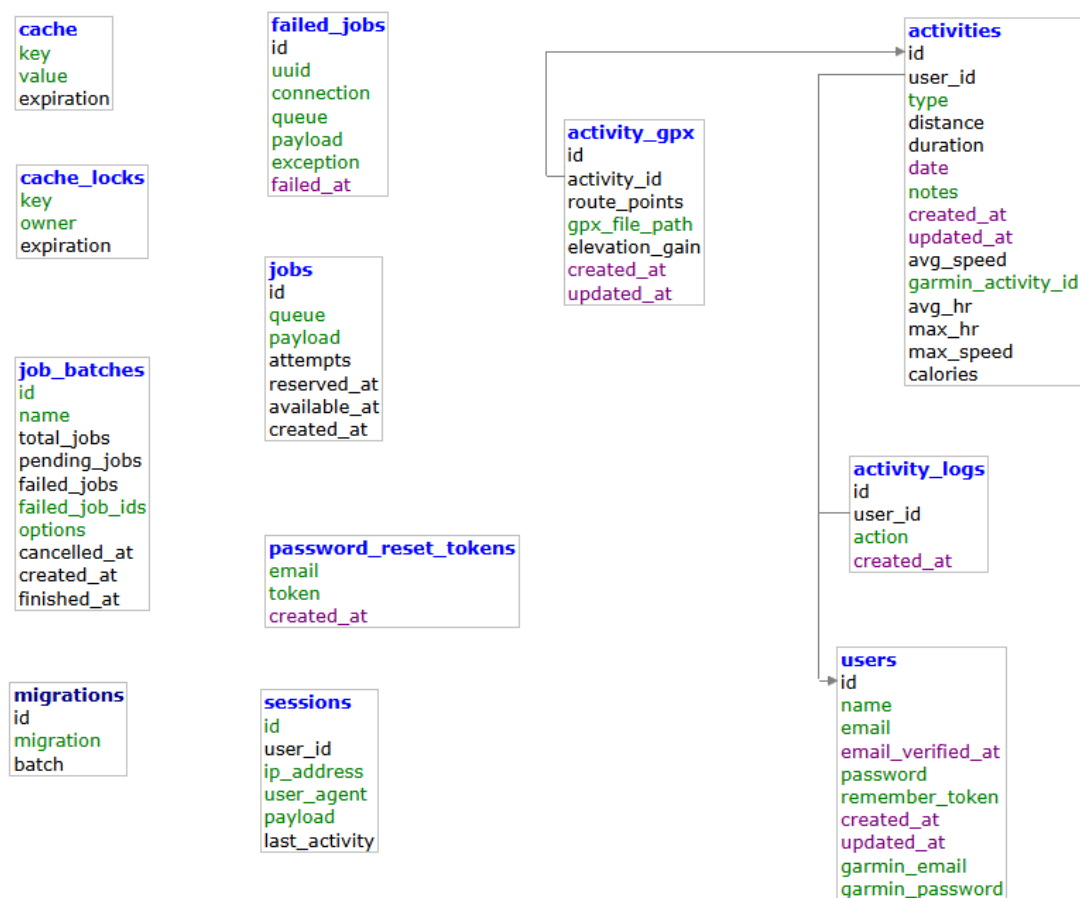
2.3.3 Tabela cache

Bufor danych (driver: database) – pary klucz–wartość z czasem wygasania.

2.3.4 Tabela password_reset_tokens

Jednokrotne tokeny resetowania hasła powiązane z adresem e-mail.

2.4 Diagram relacji encji



Rysunek 2.1: Diagram relacji encji bazy danych

2.5 Więzy integralności

2.5.1 Klucze obce i usuwanie kaskadowe

Wszystkie relacje między tabelami aplikacyjnymi zabezpieczone są kluczami obcymi z kaskadowym usuwaniem (ON DELETE CASCADE). Oznacza to, że usunięcie rekordu nadrzęd-

nego automatycznie usuwa wszystkie powiązane rekordy podrzędne – nie ma możliwości pozostawienia osieroconych danych.

Tabela	Kolumna FK	Ref. tabela	Zachowanie przy usunięciu
activities	user_id	users.id	CASCADE – usunięcie użytkownika usuwa wszystkie jego aktywności
activity_gpx	activity_id	activities.id	CASCADE – usunięcie aktywności usuwa dane trasy GPS
activity_logs	user_id	users.id	CASCADE – usunięcie użytkownika usuwa jego logi
sessions	user_id	users.id	SET NULL – usunięcie użytkownika unieważnia sesje

Dzięki takiej konfiguracji operacja **DELETE** na tabeli **users** jest bezpieczna i atomowa – baza danych gwarantuje spójność bez konieczności ręcznego usuwania powiązanych rekordów w kodzie aplikacji.

2.5.2 Ograniczenia unikalności

Tabela	Kolumna	Uzasadnienie
users	email	Każdy adres e-mail może być przypisany do jednego konta
activity_gpx	activity_id	Jedna aktywność może mieć co najwyżej jeden rekord trasy GPS
failed_jobs	uuid	Unikalny identyfikator nieudanego zadania kolejki

2.5.3 Indeksy

Poniżej zestawiono indeksy założone na kolumnach najczęściej używanych w klauzulach **WHERE** i **ORDER BY**:

Tabela	Indeksowane kolumny	Cel
users	email (UNIQUE)	Wyszukiwanie użytkownika podczas logowania
activity_logs	action	Filtrowanie logów po typie akcji
activity_logs	(user_id, created_at)	Złożony indeks – pobieranie logów użytkownika posortowanych chronologicznie
sessions	user_id	Wyszukiwanie sesji należących do użytkownika
sessions	last_activity	Czyszczenie wygasłych sesji
cache	expiration	Czyszczenie wygasłych wpisów cache
jobs	queue	Pobieranie zadań z konkretnej kolejki

2.6 Iteracyjny rozwój schematu bazy danych

Schemat bazy danych powstawał iteracyjnie – każda zmiana struktury to osobny, wersjonowany plik migracji. Podejście to gwarantuje pełną historię zmian w repozytorium Git oraz możliwość odtworzenia dowolnego stanu schematu przez uruchomienie migracji w górę (`migrate`) lub w dół (`migrate:rollback`).

Data	Plik migracji	Zmiana
2026-03-30	<code>create_activities_table</code>	Pierwsza wersja tabeli <code>activities</code> z kolumnami bazowymi i kolumną <code>weather_json</code> (JSON)
2026-04-29	<code>create_activity_gpx_table</code>	Nowa tabela <code>activity_gpx</code> przechowująca punkty trasy GPS; FK do <code>activities</code> z CASCADE
2026-04-29	<code>update_activities_table</code>	Usunięcie <code>weather_json</code> (dane pogodowe poza zakresem projektu); dodanie <code>avg_speed</code> – normalizacja schematu
2026-05-02	<code>create_activity_logs_table</code>	Tabela <code>activity_logs</code> (immutable log zdarzeń) z indeksami na <code>action</code> i (<code>user_id</code> , <code>created_at</code>)
2026-05-07	<code>add_garmin_activity_id</code>	Dodanie <code>garmin_activity_id</code> do <code>activities</code> – klucz obcy do Garmin Connect, zapobiega duplikatom przy synchronizacji
2026-05-07	<code>add_stats_to_activities</code>	Dodanie kolumn <code>avg_hr</code> , <code>max_hr</code> , <code>max_speed</code> , <code>calories</code> – rozszerzenie danych ze źródeł Garmin
2026-05-07	<code>add_garmin_credentials</code>	Dodanie <code>garmin_email</code> i <code>garmin_password</code> do tabeli <code>users</code> – przechowywanie zaszyfrowanych danych Garmin

Każdy plik migracji implementuje metody `up()` (aplikacja zmiany) i `down()` (cofnięcie zmiany), co umożliwia pełny rollback do dowolnego punktu w historii schematu.

2.7 Seedery i dane testowe

2.7.1 DatabaseSeeder

Seeder wypełnia bazę danych wygenerowanymi danymi testowymi, umożliwiając szybkie uruchomienie aplikacji z realistycznym zestawem rekordów bez ręcznego wprowadzania danych.

Działanie `DatabaseSeeder::run()`:

1. Tworzy (lub odnajduje) stałego użytkownika demonstracyjnego `nadia@doe.com` z hasłem `password`.
2. Generuje 50 aktywności powiązanych z tym użytkownikiem.
3. Tworzy 100 losowych użytkowników przy użyciu `UserFactory`.
4. Generuje 1000 dodatkowych aktywności przypisywanych losowo do wszystkich 101 użytkowników.

Po uruchomieniu seeda baza zawiera łącznie **101 użytkowników** i **1050 aktywności**.

2.7.2 ActivityFactory

`ActivityFactory` generuje realistyczne dane aktywności z użyciem biblioteki `Faker`:

Kolumna	Generowana wartość
type	losowo: running / cycling / swimming
distance	losowa liczba zmiennoprzecinkowa 0,1–100 km
duration	losowa liczba całkowita 10–300 minut
date	losowa data
notes	losowe zdanie w języku angielskim

2.7.3 Uruchomienie seederów

```
# Samo uruchomienie seeder w (bez czyszczenia bazy)
php artisan db:seed

# Czyszczenie bazy i ponowna migracja + seed (rodowisko deweloperskie)
php artisan migrate:fresh --seed
```

2.8 Testowanie i optymalizacja

5. Testowanie poprawności operacji CRUD

Każda z czterech operacji na aktywności została zweryfikowana manualnie:

- **Create** – ręczne dodanie aktywności przez formularz; weryfikacja zapisu w bazie i wyświetlenia na liście aktywności,
- **Read** – przeglądanie listy aktywności z filtrowaniem po typie i paginacją; weryfikacja poprawności wyświetlanych danych,
- **Update** – edycja danych aktywności; weryfikacja zapisu zmian,
- **Delete** – usunięcie aktywności; weryfikacja kaskadowego usunięcia rekordu z tabeli `activity_gpx` oraz pojawienia się wpisu w `activity_logs`.

Testy przeprowadzono również dla importu z pliku GPX i synchronizacji Garmin, weryfikując poprawność parsowania danych i wykrywania duplikatów na podstawie `garmin_activity_id`.

6. Optymalizacja zapytań przez indeksy

Najczęściej wykonywane zapytania zostały zabezpieczone indeksami na poziomie migracji:

- `users.email` – indeks unikalny przyspiesza wyszukiwanie użytkownika podczas logowania (operacja `WHERE email = ?`),
- `activity_logs(user_id, created_at)` – złożony indeks pokrywający zapytania pobierające logi konkretnego użytkownika posortowane chronologicznie,
- `activity_logs(action)` – indeks filtrowania logów po typie akcji.

Kolumna `activities.user_id` korzysta z indeksu automatycznie tworzonego przez Laravel przy definicji klucza obcego (`foreignId`), co przyspiesza scopowanie zapytań do zalogowanego użytkownika (`WHERE user_id = ?`).

7. Normalizacja schematu

W trakcie iteracyjnego rozwoju projektu przeprowadzono normalizację schematu: dane trasy GPS wydzielono do osobnej tabeli `activity_gpx` (zamiast przechowywania w kolumnie JSON w tabeli `activities`). Dzięki temu zapytania do listy aktywności nie pobierają potencjalnie dużych danych trasy – relacja `gpxData()` ładowana jest wyłącznie na żądanie (*lazy loading*) przy wyświetleniu szczegółów aktywności. Eliminuje to problem N+1 na liście zbiorczej.